

StratRoom

Task & Initiative Progress Update Module

Chat Agent Integration & Intent Routing

A structured technical report on the progress-update tooling, its automated intent routing in the Chat Engine, RBAC scoping, data model, verification results, and deployment steps.

Prepared August 2026 • Version 1.0 • Internal documentation

Contents

- 1. Executive summary
- 2. Scope & terminology
- 3. Architecture at a glance
- 4. The Progress Update Module (task_details / initiatives_details)
- 5. Chat Agent Integration (end-to-end flow)
- 6. Intent detection & routing (_auto_fetch_tool)
- 7. Tool execution (_execute_tool) & RBAC
- 8. Direct API endpoints (task-action / initiative-action)
- 9. Provider resolution & configuration
- 10. Verification & test results
- 11. Deployment
- 12. Troubleshooting & gotchas
- 13. Appendix — reference tables, regexes, SQL

1. Executive summary

This report documents the **Task & Initiative Progress Update Module** and its integration with the **StratRoom Chat Agent**. The module lets users update the progress percentage of **tasks** (stored in task_details) and **initiatives / projects** (stored in initiatives_details) either directly through dedicated API endpoints or through natural-language chat.

A focused hardening pass was applied in August 2026 after a production incident in which the phrase "update initiative 8 progress to 80%" erroneously updated **Task #8** instead of **Initiative SI-01**. The root cause was that initiative/project intent detection was gated to the projects and strategy agents only, so in the **tasks** chat the query fell through to the LLM which mis-identified it as a task. The fix makes initiative/project detection **agent-agnostic** and adds prompt-level guardrails. All changes were deployed to production and verified live.

Key facts	Detail
Feature area	Progress update for tasks (0-100%) and initiatives (0-100%), query + update
Backend endpoints	POST /agents/chat, POST /agents/task-action, POST /agents/initiative-action, GET /agents/effective-config
Key files	app/agents/base.py, prompts.py, task_tools.py, initiative_tools.py; app/routers/agents.py
Persistence	MySQL schema orgstructure; JSON columns task_value / initiative_value + status columns
RBAC model	admin = unscoped; member = owner + org scoped (JOIN employee_details)
Clamping	Progress clamped to 0-100 in both tools; confirmation reports effective value

Key facts	Detail
Incident fixed	Initiative intent now detected for ALL agents; task prompt warns initiative != task
Deployment	docker cp backend files into stratroom_api + docker restart; or deploy.py full build

2. Scope & terminology

2.1 In scope

- **update_task_progress()** — writes the progress key into task_details.task_value and auto-derives task status.
- **update_initiative_progress()** — writes progressval/progress/statusLight/statusIndicator into initiatives_details.initiative_value.
- **query_user_tasks()** / **query_initiatives()** — RBAC-scoped reads used by chat and the API endpoints.
- **_auto_fetch_tool()** — the pre-LLM intent rewriter that routes initiative/task phrasing correctly.
- **_execute_tool()** — the dispatch table that turns [TOOL_CALL:name:args] into a Python call.
- The **/agents/chat** request path, **/agents/task-action** and **/agents/initiative-action** direct endpoints.

2.2 Terminology

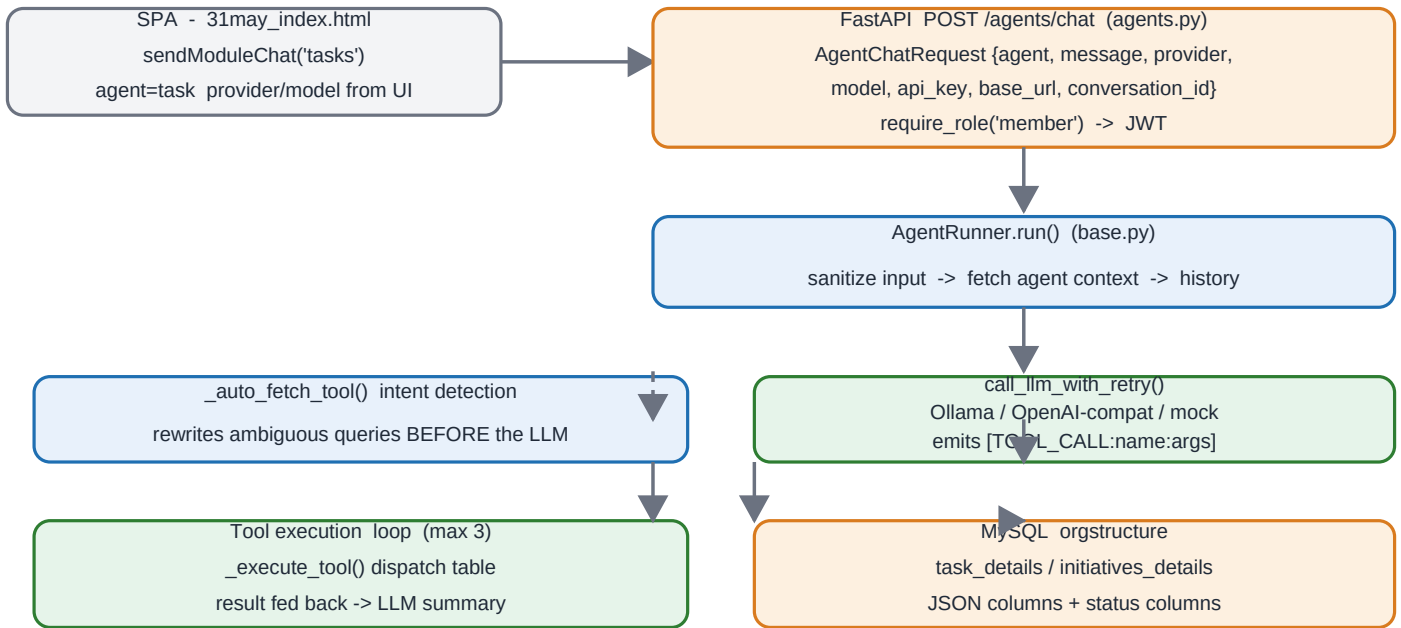
Term	Meaning
Task	A work unit in task_details; has progress, status, priority, owner, due_date.
Initiative / Project	A strategic initiative in initiatives_details; has initiative_id (e.g. SI-01), progressval, statusLight, statusIndicator.
Agent	An LLM persona with its own system prompt (task, strategy, projects, risk, ...). Chat selects it via the moduleId->agent map.
Tool call	LLM-issued marker [TOOL_CALL:name:arg=val,...] that the runner executes and feeds back.
RBAC	Role-based access control: admin vs member scoping applied in every tool.
Bridge	JavaBridge wrapper for MySQL I/O (bridge._mysql / bridge._mysql_write), used by all tools.

3. Architecture at a glance

The single-file SPA (31may_index.html) sends module chats to POST /agents/chat. Apache :8088 proxies to FastAPI :8001 inside container stratroom_api. The agent endpoint builds an AgentRunner for the requested agent domain, runs intent detection against the user message, calls the configured LLM, executes any emitted tool call through _execute_tool(), and returns a natural-language reply plus tool_results.

All progress writes go to **MySQL schema orgstructure**. The JSON columns task_details.task_value and initiatives_details.initiative_value hold the rich record; status / priority columns mirror the most important facets for faster indexing and the legacy UI.

FLOW CHART 1 — Chat Agent integration: front-end -> /agents/chat -> tool execution



The tool-calling loop runs up to 3 iterations: execute the tool, then ask the LLM to summarize the result naturally.

Chart 1 - The full chat path. sendModuleChat() maps a UI module to an agent domain, posts to /agents/chat, and the runner rewrites the query, calls the LLM, and executes tool calls against MySQL.

4. The Progress Update Module

4.1 Data model

Table	Key columns
task_details	ID, task_value (JSON), status, priority, owner, source_module, page_name, updated_time, created_time
initiatives_details	id, initiative_id (SI-01), initiative_value (JSON), owner, updated_time, updated_by

task_value carries keys such as Name/title, progress, status, dueDate, assignedUserId, sourceModule, agent. initiative_value carries name, description, progressval, progress, statusLight (CSS classes), statusIndicator (GREEN/AMBER/RED), ownerName. Both are edited **read-modify-write** so unrelated keys survive.

4.2 Progress semantics & auto-status

Range	Derived value	Confirmation wording
progress = 0	status = pending	Task remains Pending
0 < progress < 100	status = in_progress	Task marked as In Progress
progress >= 100	status = completed	Task auto-marked as Completed
initiative: 0-39	statusIndicator = RED	statusLight = progress-bar-danger
initiative: 40-74	statusIndicator = AMBER	statusLight = progress-bar-warning
initiative: 75-100	statusIndicator = GREEN	statusLight = progress-bar-success

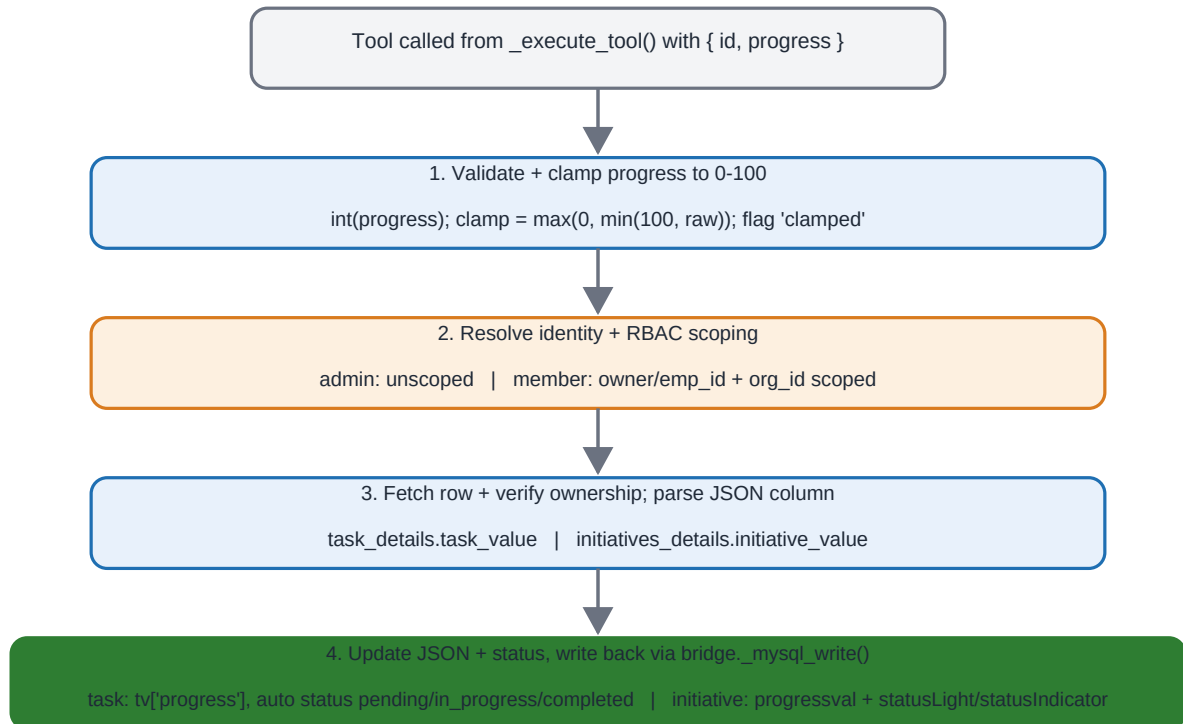
Both tools clamp the requested value to **0-100**. If the raw value was out of range (e.g. 305), the confirmation string explicitly states "requested value X was out of range and has been clamped to Y% (progress must be 0-100)" so the LLM and the user see the effective DB value.

4.3 RBAC scoping

- Admin:** updates are unscoped — single-table UPDATE by ID (business data spans orgs while admins sit in org 1).

- **Member:** identity resolved from email -> employee_details; writes are scoped with AND owner = emp_id (tasks also JOIN employee_details on org_id).
- Ownership mismatch -> "This task/initiative is not yours."; missing row -> "...not found."

FLOW CHART 3 — update_task_progress() / update_initiative_progress() write path



All other JSON keys are preserved (read-modify-write). Confirmation string reports the effective value incl. clamping.

Chart 3 - The write path for both progress tools: clamp -> identity/RBAC -> ownership check -> read-modify-write JSON -> status derivation -> confirmation.

5. Chat Agent Integration (end-to-end)

5.1 Front-end request

```
agentMap = { risk:'risk', scorecard:'strategy', budget:'finance', tasks:'task', meetings:'meetings',
projects:'projects', incidents:'incident', compliance:'compliance', audit:'audit', swot:'strategy',
pestel:'strategy', forecast:'strategy', docintel:'strategy', org:'strategy', bcp:'compliance' }
fetch('/agents/chat', { method: 'POST', headers:
{'Content-Type':'application/json','Authorization':'Bearer '+getToken()}, body: JSON.stringify({
agent: agentMap[moduleId], // e.g. 'task' message: userMsg, provider: cfg.provider, // e.g. 'ollama'
api_key: cfg.apiKey || '', model: cfg.model || 'llama3.2', conversation_id:
_agentConversations[agentDomain] || null, base_url: cfg.baseUrl }) });
```

The browser aborts the request after **45 s** (AbortController) and shows a timeout message if no reply arrives. Provider/model are read from the UI LLM config (`_getLLMConfig()`); the response renders AI bubbles plus interactive task/initiative cards from `tool_results`.

5.2 Server-side processing (AgentRunner.run)

#	Stage	Responsibility
1	sanitize_input()	Enforce inbound length / character restrictions on the user message.
2	fetch_agent_context()	Load CURRENT ORGANIZATION DATA for the agent's modules.
3	enhance_context() / history	Attach AI memories and the conversation history.
4	_auto_fetch_tool()	Rewrite confusing queries (see section 6) BEFORE calling the LLM.
5	call_llm_with_retry()	Call the configured provider with system prompt + context + message.
6	Tool-calling loop (max 3)	Parse [TOOL_CALL:...], execute via _execute_tool(), feed result back, ask LLM to summarize.
7	Persistence + metrics	Log agent run, store conversation + messages, token benchmark.

5.3 Tool-call pattern

```
# Regex used by the runner (base.py:31) _TOOL_CALL_RE =
re.compile(r"\[TOOL_CALL:([^\]]:)+\](?:([^\]]*)?)\]") # Example emitted by the LLM and executed by
_execute_tool(): [TOOL_CALL:update_initiative_progress:id=8,progress=80]
[TOOL_CALL:update_task_progress:id=102,progress=60] [TOOL_CALL:query_initiatives] # kv parser
accepts id= or task_id=/initiative_id= aliases and int-casts values.
```

The runner feeds the tool result back to the LLM with the instruction to summarize it naturally and, for updates, to confirm the change was saved. Tool results also travel in the JSON response so the SPA can render rich cards.

6. Intent detection & routing (_auto_fetch_tool)

`_auto_fetch_tool()` (base.py) runs before the LLM and rewrites ambiguous queries into precise, self-contained instructions. It is the primary defence that prevents initiative/project phrasing from being mis-routed to task tools.

6.1 Routing table

Example phrase	Intent	Rewrite target
update initiative 8 progress to 80%	INITIATIVE	Call update_initiative_progress with id=8, progress=80.
set project 12 to 60%	INITIATIVE	Call update_initiative_progress with id=12, progress=60.
update the progress of initiative SI-02 to 75%	INITIATIVE	SI- prefix is accepted (lowercased). id=2, progress=75.
update initiative progress to 80% for Market Analysis	NAME-ONLY (ASK ID)	Politely ask for the numeric initiative ID; never touch a task.
update progress of task 102 to 80%	TASK	Call update_task_progress with id=102, progress=80.
set task 5 progress to 60%	TASK	Call update_task_progress with id=5, progress=60.
mark task 102 as completed	TASK STATUS	Call update_task_status with id=102, status=completed.
show me all initiatives	QUERY	Call query_initiatives (projects/strategy agents).

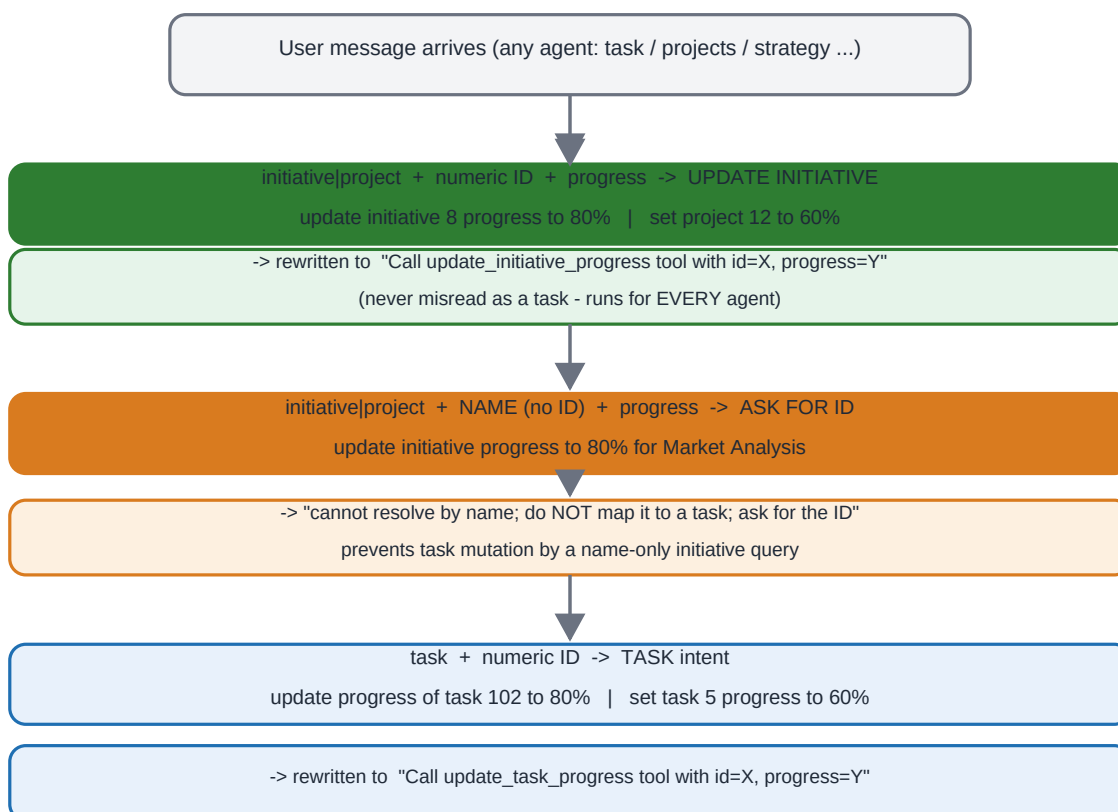
Example phrase	Intent	Rewrite target
list all tasks	QUERY	Call query_tasks.

The initiative/project block is evaluated **before** the task block and is **not gated by agent name** — so the same phrase routes to initiative tools from the tasks chat, the projects chat, or any other module chat. Task detection then only ever fires for phrases that explicitly name a **task**.

6.2 Prompt-level guardrail (task agent)

The task system prompt (prompts.py) now states: *"The words 'initiative' and 'project' refer to the Initiatives module, NOT to tasks ... NEVER call a task tool. Instead call update_initiative_progress ... If the user only names an initiative without a numeric ID, politely ask for the numeric initiative ID rather than guessing a task."* The prompt also documents update_initiative_progress and query_initiatives as valid tools for the task agent.

FLOW CHART 2 — Intent detection & routing in _auto_fetch_tool()



Detection order: initiative/project block is evaluated BEFORE the task block for every agent.

Chart 2 - Intent routing: initiative/project detection runs for every agent, name-only references are intercepted, and only explicit task phrasing reaches the task tools.

7. Tool execution (_execute_tool) & RBAC

_execute_tool(db, tool_name, args_raw, user_id, org_id, is_admin, ctx) parses the kv arguments and dispatches to the matching Python tool. The full registry advertised in the error message is: query_tasks, update_task, update_task_status, update_task_progress, create_task, create_risk_mitigation_task, query_initiatives, update_initiative_progress, update_incident_status, query_decisions, create_decision, update_decision_status, query_risks, create_risk, update_risk, delete_risk, query_scorecards, query_scorecard_summary, create_scorecard, update_scorecard, delete_scorecard.

Tool name	Python call	Kind
query_tasks	query_user_tasks(db, user_id, org_id, status, priority, owner, is_admin, email)	Read, RBAC-scoped, self_only=True for chat

Tool name	Python call	Kind
update_task_progress	update_task_progress(db, user_id, org_id, task_id, progress, is_admin, email)	Write, clamp 0-100, auto status
update_task_status	update_task_status(db, user_id, org_id, task_id, status, is_admin, email)	Write, normalized status
query_initiatives	query_initiatives(db, user_id, org_id, is_admin, email, owner, status)	Read, RBAC-scoped
update_initiative_progress	update_initiative_progress(db, user_id, org_id, initiative_id, progress, is_admin, email)	Write, clamp 0-100, statusLight/statusIndicator

- ID aliases: `id=`, `task_id=`, `initiative_id=` all resolve via `_tid()`.
- Progress values are int-cast; non-integers return an error with usage guidance.
- All writes go through `bridge._mysql_write()` with parameterized queries (no string-injected SQL).

8. Direct API endpoints (no LLM)

In addition to chat, two direct endpoints execute the same tools without an LLM round trip. These are used by the SPA for explicit buttons and by scripts.

Endpoint	Request body	Backing functions
POST /agents/task-action	<code>{"action": "query" "update" "update_progress", "task_id": N, "status": "...", "progress": N}</code>	query_user_tasks / update_task_status / update_task_progress
POST /agents/initiative-action	<code>{"action": "query" "update_progress", "initiative_id": N, "progress": N}</code>	query_initiatives / update_initiative_progress
GET /agents/effective-config	—	Server default provider/model + key_configured flag for the provider badge

Both write endpoints require `require_role('member')` and use the caller's RBAC context (`is_admin`, `org_id`, `email`) just like the chat path. Errors map to 422 (missing fields) or 404 (not found / not yours).

9. Provider resolution & configuration

The chat endpoint resolves the LLM provider from the request, falling back to server defaults when the client supplies neither an `api_key` nor a `base_url`.

Condition	Resolution
Client supplied <code>api_key</code> or <code>base_url</code>	provider = req.provider or server default; <code>api_key</code> = req. <code>api_key</code> or server default
Client supplied neither	provider = server <code>AI_PROVIDER</code> or req.provider; <code>api_key</code> = server <code>AI_API_KEY</code>
ollama	No API key required; endpoint = <code>base_url/api_key</code> override or default (localhost:11434)
mock	Deterministic canned reply for tests; still exercises the full HTTP + routing path
Provider badge	GET /agents/effective-config returns server defaults so the UI shows e.g. 'TOGETHER' instead of hardcoded text

Allowed providers (`settings.ALLOWED_PROVIDERS`): openai, anthropic, google, deepseek, moonshot, together, mistral, xai, ollama, mock.

10. Verification & test results

10.1 Automated tests

Suite	Result	Coverage
tests/test_initiative_tools.py	15/15 PASS	progress parser (clamp low/high/bounds + new 3-tuple), status light/indicator, query RBAC, update ownership, not-found, missing-value, owner-scoped write
tests/test_suite.py	124/128 PASS	Full offline suite. 4 failures are pre-existing and unrelated (PostgreSQL removal in Jul 2026): test_import_db, docker-compose PG, asyncpg requirement.

10.2 Live production verification (August 2026)

Check	Result	Evidence
Restore	Task #8 'Deploy Fraud Detection...' => 100 / completed; Task #32 'market research' => 0 / pending	Confirmed via POST /agents/task-action query
Routing	'update initiative 8 progress to 80%' from task agent -> routes to update_initiative_progress:id=8,progress=80 (NOT update_task_progress)	Verified in container with AgentRunner._auto_fetch_tool
Routing	'set project 9 to 60%' -> update_initiative_progress:id=9	Verified in container
Routing	'update progress of task 102 to 80%' -> update_task_progress:id=102	Unchanged/regression-free
Name-only	'update initiative progress to 80% for Market Analysis' -> asks for numeric ID, never mutates a task	Verified in container
No mutation	POST /agents/chat with provider=mock for all three failing queries left Task #8 and #32 untouched	tasks untouched == True
Deployed signals	base.py (agent-agnostic block), prompts.py (guardrails + tools), task_tools.py / initiative_tools.py (clamp feedback)	grep + python import OK inside stratroom_api

11. Deployment

Two deploy styles are in use. For this change only three backend files changed (base.py, prompts.py, task_tools.py, initiative_tools.py), so the hot-swap path was used.

11.1 Hot-swap (fast, for backend-only file changes)

```
# 1. copy files into the build source on the host scp backend/app/agents/base.py
root@103.191.132.36:/opt/stratroom-new/backend/app/agents/ # ... repeat for prompts.py,
task_tools.py, initiative_tools.py # 2. copy into the running container + restart
(PYTHONPATH=/app/backend) docker cp /opt/stratroom-new/backend/app/agents/base.py
stratroom_api:/app/backend/app/agents/base.py docker restart stratroom_api # 3. verify curl -s -m 5
http://localhost:8001/health docker exec stratroom_api python -c "from app.agents.base import
AgentRunner; print('ok!')"
```

11.2 Full build (deploy.py)

deploy.py copies 41 files, backs them up, rebuilds the API container with `docker compose up -d --no-deps --build api`, copies the SPA to the Apache doc root, waits, and health-checks. It exits non-zero on any failure. SSH password is read from `os.environ['SSH_PASS']`.

```
$env:SSH_PASS=''; python deploy.py
```

12. Troubleshooting & gotchas

- **"update initiative X" touched a task in chat.** Ensure the deployed base.py contains the agent-agnostic block; if the container hostname differs from stratroom_api, apply `docker cp` to the real name (docker ps).
- **Chat times out after 45 s.** Usually a slow or unreachable LLM. With intent detection the query is rewritten before the LLM, which cuts most reduceable latency; check /agents/status for provider health.

- **Provider badge shows the wrong provider.** GET /agents/effective-config surfaces server defaults {provider, model, key_configured}; the SPA prefers client-side LLM config.
- **Progress made up.** Both progress tools clamp to 0-100 and report the effective value, so 305 is stored as 100 with an explicit clamping note.
- **Member can update a task they don't own?** No — writes are scoped by owner/emp_id (+ org_id for tasks) and return a 'not yours' error.
- **SI-02 phrases not detected.** Now handled: the regex accepts a lowercased si- prefix before the numeric ID.
- **JSON keys lost on update?** They should not be — tools read the column, mutate only progress-related keys, and write the whole JSON back.
- **Rate limiter.** In-memory per-process; container must run --workers 1 (120 req/min global, 30/min AI).
- **JWT_SECRET ephemeral.** If unset it is auto-generated and regenerated on restart; keep a static 64-char hex in production.

13. Appendix

13.1 Intent-detection regexes (base.py auto fetch tool)

```
p1 = r"(?:update|set|change)\s+(?:the\s+)?progress\s+(?:of|for)?\s+"
r"(?:initiative|project)\s+[#:=s]*(si-)?(\d+)\s+to\s+(\d+)" p2 =
r"(?:update|set|change)\s+(?:initiative|project)\s+[#:=s]*(si-)?(\d+)\s+" r"progress\s+to\s+(\d+)"
p3 = r"(?:update|set|change)\s+(?:initiative|project)\s+[#:=s]*(si-)?(\d+)\s+" r"to\s+(\d+)" #
name-only guard (no numeric ID): if
re.search(r"(?:update|set|change).*(?:initiative|project).*progress", msg_lower.replace("%", ""))
and \ not re.search(r"(?:initiative|project)\s+[#:=s]*\d+", msg_lower): return "...ask for the
numeric initiative ID..."
```

13.2 Reference: sample SQL (admin update paths)

```
-- task progress (admin; member adds JOIN employee_details + owner/org scoping) UPDATE task_details
t SET t.task_value = %s, t.updated_time = NOW() WHERE t.ID = %s; UPDATE task_details t SET t.status
= %s WHERE t.ID = %s; -- initiative progress UPDATE initiatives_details SET initiative_value = %s,
updated_time = NOW(), updated_by = %s WHERE id = %s;
```

13.3 Confirmation-string examples

Scenario	Confirmation text
Task #8 from 100% to 80%	Successfully updated progress for Task #8 from 100% to 80% in the database. Task marked as In Progress.
Task progress 305 (clamped)	Successfully updated progress for Task #32 from 0% to 100% in the database. Note: the requested value 305 was out of range and has been clamped to 100% (progress must be 0-100). Task auto-marked as Completed.
Initiative #8 to 80%	Successfully updated progress for Initiative #8 (SI-01) from 0% to 80% in the database. Initiative status: GREEN.

13.4 Key source references

File	Symbol	Notes
base.py	_auto_fetch_tool (line ~243)	Agent-agnostic initiative intent + name-only guard, task detection
base.py	_execute_tool (line ~374)	Dispatch table incl. update_task_progress / update_initiative_progress
prompts.py	"task" prompt (line ~176)	initiative != task rule + tools documentation
task_tools.py	update_task_progress (line ~424)	clamp 0-100, status auto-derive, RBAC writes
initiative_tools.py	update_initiative_progress (line ~113)	clamp 0-100, statusLight/statusIndicator, RBAC writes
routers/agents.py	agent_chat / task-action / initiative-action	HTTP entry points, provider resolution, RBAC
frontend/31may_index.html	sendModuleChat (line ~20517)	moduleId->agent map, /agents/chat payload, 45s abort

End of report - generated automatically from the StratRoom repository (Progress_Update_Module_Report.pdf).